

Lab Report 5: I2C & SPI Driver Development and Sensor Integration

Authors:

J.M. Páez Sandoval, 37945

L.A. Lugo Muñoz, 37915

D. Lopez Moreno, 37984

C.A. Martínez Macías, 33369

Teacher : Carlos A. Villareal

CETYS Universidad

Facultad de Ingeniería

Curso: Microcontroladores y Sistemas Embebidos

Fecha: May 29, 2026

Abstract—This report documents the design and bare-metal implementation of SPI communication firmware for the NUCLEO-F411RE development board based on the STM32F411RE ARM Cortex-M4 microcontroller. The project configures the SPI peripheral to establish full-duplex serial communication with the ADXL345 three-axis digital accelerometer, enabling real-time sensor data acquisition. A modular driver architecture is implemented to separate low-level hardware access (SPI and GPIO drivers) from sensor-level functionality and application-level control, following the guidelines defined in the Software Requirements Specification (SRS). Acceleration measurements along the X, Y, and Z axes are acquired from the ADXL345 and mapped to application outputs such as PWM duty-cycle modulation or UART serial transmission, demonstrating real-time embedded-system interaction based on sensor input. The project highlights the use of hardware abstraction, CMSIS-based register configuration, and layered embedded software design for reliable SPI communication and sensor integration.

Index Terms—STM32F411RE, SPI, ADXL345, bare-metal programming, CMSIS, full-duplex communication, accelerometer, embedded systems, hardware abstraction, sensor driver.

I. INTRODUCTION

In the development of modern embedded systems, the ability to communicate with external devices through standardized serial protocols is a fundamental skill. Among the most widely used communication protocols are I2C (Inter-Integrated Circuit) and SPI (Serial Peripheral Interface), each offering characteristics that make them suitable for different applications. SPI, as a full-duplex four-wire protocol (MOSI, MISO, SCK, and CS), enables high-speed data transfers, making it an ideal choice for sensors that require fast and reliable communication.

The purpose of this lab is to design and implement an SPI driver on the STM32 Nucleo-64 microcontroller, following the guidelines established in the Software Requirements Specification (SRS) document. Based on this driver, a sensor interface is

developed for the ADXL345, a three-axis digital accelerometer that communicates through the SPI protocol and is capable of measuring acceleration along the X, Y, and Z axes with configurable resolution.

The development process follows a modular hardware abstraction approach, clearly separating responsibilities between the low-level SPI driver, the sensor driver, and the application layer. This architecture improves code reusability, scalability, and maintainability, which are essential principles in embedded systems software engineering.

As part of the system integration, acceleration data acquired from the ADXL345 is processed and mapped to an application output such as PWM duty-cycle modulation or UART serial transmission. This demonstrates the complete embedded-system workflow, from sensor data acquisition and communication to real-time processing and output control.

II. PROCESS DESCRIPTION

A. I2C Driver

The I2C communication module was created using the files `i2c_driver.h` and `i2c_driver.c`. The driver defines the register mapping for the STM32F411RE I2C peripheral, configuration structures, bit definitions, and return-status enumerations required for peripheral control. Communication is performed directly through the hardware registers described in the STM32 reference manual, including the

control, timing, status, and data registers associated with the I2C1 interface.

Several functions were implemented to support peripheral initialization and device communication. The API includes routines for register-based transactions as well as direct byte transfers without internal addressing. This design allows the driver to support a wide variety of I2C devices instead of being limited to a single sensor type.

The communication sequence required by the I2C protocol was implemented through continuous monitoring of hardware status flags. During a register-read operation, the software generates a start condition, sends the slave address and target register, performs a repeated start, switches to read mode, retrieves the requested bytes, and terminates the transfer with a stop condition. Correct synchronization between these stages is essential to avoid bus errors and incomplete transfers.

To improve reliability, timeout protection was integrated into all polling routines. If a device does not respond within the expected interval, the driver exits the operation and returns an error code instead of blocking program execution. Additional error detection was also included for conditions such as missing acknowledgments, arbitration conflicts, and bus faults.

B. MPU6050 Sensor Driver

The MPU6050 sensor driver was implemented as an abstraction layer above the I2C communication module. The driver is responsible for sensor initialization, configuration, and acquisition of accelerometer and gyroscope measurements. All hardware communication with the sensor is handled exclusively through the I2C driver, which keeps the sensor layer independent from direct peripheral register manipulation.

The initialization sequence begins by verifying communication with the sensor through its identification register. Once the device is detected successfully, the driver configures the power-management registers, selects the desired sensitivity ranges, and enables measurement operation. Additional configuration options such as accelerometer scale and gyroscope sensitivity can also be modified depending on application requirements.

Sensor data is retrieved by reading consecutive registers that contain the high-byte and low-byte values for each measurement axis. The driver combines these bytes into signed 16-bit variables representing acceleration and angular velocity along the X, Y, and Z axes. Using burst reads allows multiple measurements to be acquired efficiently within a single I2C transaction.

The driver also includes basic validation and error handling mechanisms. Communication failures, invalid responses, or timeout conditions are detected and reported through return-status codes, simplifying debugging and improving system reliability during operation.

C. Main Application

The application layer combines the I2C driver, MPU6050 sensor driver, GPIO module, and UART communication interface into a complete embedded system. Program execution is organized using a simple state-based structure responsible for sensor acquisition and serial-data transmission.

During operation, the microcontroller continuously retrieves acceleration and gyroscope measurements from the MPU6050 sensor and processes the acquired data in real time. The measured values are then transmitted to the serial terminal using the UART formatting functions developed in previous labs.

The serial output displays labeled sensor readings for each axis, allowing real-time monitoring of motion and orientation data. This implementation demonstrates the full embedded-system workflow, including low-level communication, sensor interfacing, data acquisition, and serial visualization.

To improve robustness, the application includes fault-detection mechanisms capable of identifying communication errors and timeout events. When an error occurs, descriptive messages are printed through the serial interface instead of invalid numerical values, making debugging and testing significantly easier.

III. RESULTS

After programming the STM32F411RE board and opening the VS Code Serial Monitor at 115200 baud, the system initialized correctly and established communication with the MPU6050 sensor through the I2C interface. During startup, the firmware verified the sensor connection by reading the device identification register, confirming that the MPU6050 was properly connected and responding on the bus. Once initialization was completed, the application continuously acquired acceleration and gyroscope measurements from the sensor and transmitted the data to the serial terminal in real time.

The MPU6050 module was connected to the NUCLEO-F411RE board through the I2C1 interface using four main connections: SDA, SCL, VCC, and GND. The sensor operated at 3.3 V and shared a common ground reference with the microcontroller board. During testing, the development board was moved and rotated manually in order to observe changes in acceleration and angular velocity measurements along the X, Y, and Z axes.



Fig. 1. Physical setup

The serial-monitor output obtained during operation is shown in Figure 3. The displayed values correspond to the accelerometer and gyroscope measurements acquired from the sensor. When the board remained stationary, the accelerometer readings showed small fluctuations caused by sensor noise and environmental vibrations. However, when the board was tilted, rotated, or moved abruptly, significant changes were observed in the corresponding axis values. The gyroscope measurements also responded immediately to rotational motion, confirming correct sensor operation and real-time data acquisition.

The communication routines operated reliably throughout testing. The implemented I2C driver successfully generated the required start conditions, repeated-start sequences, acknowledgment handling, and stop conditions necessary for stable data transfers. No communication failures were observed during normal operation, and the timeout protection mechanisms prevented the firmware from becoming trapped in indefinite polling loops when testing invalid or disconnected configurations.

One of the main challenges during development involved correctly interpreting the MPU6050 register map and combining the high-byte and low-byte values into

signed 16-bit measurements. Initial tests produced inconsistent values due to incorrect byte ordering, but after adjusting the register-read sequence according to the datasheet specifications, the measurements became stable and consistent with the physical motion applied to the sensor.

The modular structure of the software simplified debugging and integration considerably. Since the I2C driver, MPU6050 driver, UART routines, and application layer were implemented independently, each subsystem could be validated separately before complete system integration. All source files, including communication drivers, sensor drivers, UART modules, GPIO configuration, and application code, were maintained in the project repository as part of the final deliverable.

Functional Compliance Summary

F R	Require ment	Implementation	Stat us
F R 1	Enable I2C1 APB1 clock	RCC->APB1ENR = I2C1EN	✓
F R 2	Configure CCR for 100 kHz	CCR = 80	✓
F R 3	Generate START condition	CR1 = I2C_CR1_START	✓
F R 4	Read 14-byte burst from MPU6050	i2c_readRegDevice(0x68, 0x3B, buf, 14)	✓
F R 5	SPI1 Master Mode 3 @ ~3 MHz	CR1: MSTR SSM SSI CPOL CPHA BR/32	✓

F R 6	Software CS control	spi_cs_enable/disable via PA9	✓
F R 7	Display gyro data on LCD	lcd_print_string on rows 0 and 1	✓

TABLE I — Functional Requirements Compliance

IV. CONCLUSIONS

This project demonstrated the importance of reliable communication protocols in embedded systems and how low-level peripheral control directly affects the behavior of higher-level applications. By implementing the I2C driver from scratch and integrating it with the MPU6050 motion sensor, the system was able to acquire and process real-time acceleration and gyroscope data successfully on the STM32F411RE platform.

One of the main lessons learned during development was the importance of carefully following the device datasheet and communication sequence requirements. Even small mistakes in register configuration, byte ordering, or timing produced incorrect sensor readings during the initial stages of testing. Correctly combining high-byte and low-byte register values, configuring the power-management registers, and validating the sensor ID were essential steps to achieve stable and reliable operation.

The modular architecture used throughout the project also proved highly effective. Separating the I2C communication layer, sensor driver, UART routines, and application logic simplified debugging and allowed each subsystem to be tested independently before full integration. Reusing previously developed GPIO and UART modules reduced development time and demonstrated the advantages of hardware abstraction and layered software design in embedded applications.

Another important outcome of the project was the implementation of error-handling and timeout mechanisms. These protections prevented the firmware from becoming trapped in infinite polling loops during communication failures and improved overall system robustness. The ability to detect and report faults through the serial terminal made debugging significantly easier during hardware integration and testing.

The final implementation successfully demonstrated real-time motion-data acquisition and serial visualization using the MPU6050 sensor. The developed communication and sensor drivers can now serve as reusable foundations for future embedded projects involving external peripherals and motion-based applications. Future improvements could include sensor calibration, digital filtering, interrupt-driven acquisition, or integration with additional peripherals such as displays, wireless modules, or control systems.

Overall, the project provided practical experience in bare-metal embedded programming, peripheral configuration, sensor interfacing, and modular software development using the STM32F411RE microcontroller platform.

Github links:

1. **Daniel:**<https://github.com/lopezdaniel-web/MSE-Lab>
2. **Paez:**<https://github.com/JPaez725/MSE-Lab>
3. **Cristian:**<https://github.com/CrisxCross15/MSE-Lab>
4. **Luis:**<https://github.com/alugobji/MSE-Lab/tree/main/Lugo>

REFERENCES

- [1] D. V. Lindberg and H. K. H. Lee, "Optimization under constraints by applying an

asymmetric entropy measure," *J. Comput. Graph. Statist.*, vol. 24, no. 2, pp. 379–393, Jun. 2015, doi: 10.1080/10618600.2014.901225.

[2] B. Rieder, *Engines of Order: A Mechanology of Algorithmic Techniques*. Amsterdam, Netherlands: Amsterdam Univ. Press, 2020.

[3] I. Boglaev, "A numerical method for solving nonlinear integro-differential equations of Fredholm type," *J. Comput. Math.*, vol. 34, no. 3, pp. 262–284, May 2016, doi: 10.4208/jcm.1512-m2015-0241.